Current Workplan

Page • Tag

Complete Project Execution Plan

Current Assets (What You Have)

✅ Stage 1 - Complete and Working

- 2D dynamical system: $\dot{x}$, $\dot{T}$ equations
- Closed-form results: g^*, $T^*(g)$, $F(g)$, $x^*(g)$
- HTML interactive visualization (excellent quality)
- Inversion mechanism demonstrated
- Comparative statics identified

✅ Stage 2 - Partially Complete

- 3D system with endogenous enforcement
- $\dot{G} = \psi(x - x_{target})$ feedback rule
- Some Hopf bifurcation analysis
- Basic stability conditions

✅ Supporting Materials

- Conceptual framework document (strong theoretical motivation)
- Literature review (novelty confirmed)
- Understanding of empirical applications

❌ What's Missing

- Formal game-theoretic foundations
- Rigorous proofs for all propositions
- Complete bifurcation analysis (Stage 2)

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

Execution Plan: 6-Week Sprint to Professor Presentation

Week 1: Game-Theoretic Foundations + Theorem Formalization

Goal: Reframe existing results as equilibrium outcomes of a well-defined game

Day 1-2: Game Specification

Task: Write formal game setup that generates current dynamics

Deliverable: Mathematical document containing:

```
Plain Text

GAME SETUP:
- Players: Continuum of users (mass 1) + Platform
- User i's strategy: a_i ∈ {0,1} (share harmful content or not)
- Platform's strategy: g ∈ [0, g_max] (enforcement intensity)
- State variable: T ∈ [0,1] (institutional trust)

USER PAYOFF:
π_i(a_i, a_{-i}, g, T) = a_i · [θ_i + α·x - g·T^κ]
where x = ∫ a_j dj (population share)

POPULATION DYNAMICS (Evolutionary Game):
Users sample and imitate. Replicator dynamics:
ẋ = σx(1-x)[E[θ_i] + αx - gT^κ]

PLATFORM OBJECTIVE:
V = ∫₀^∞ e^{-ρt}[-c_x·x_t - c_g·g_t + v(T_t)]dt

TRUST EVOLUTION (Endogenous):
Ṫ = -β·g·[1 + η·|x - x_target|/x_target] + γ(T_0 - T)
Interpretation: Trust erodes faster when enforcement is visible AND
failing
```

Action Items:

- ◯ Write complete LaTeX document (5-8 pages)
- ◯ Derive replicator dynamics from evolutionary game
- ◯ Prove that user best-response generates the ẋ equation

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

nal Statements + Proofs)

rous propositions with complete proofs

Proposition 1 (Equilibrium Trust)

Plain Text

```
Statement: For fixed g, the unique equilibrium trust level is
T*(g) = T_0 - (β/γ)·g
provided g ≤ γT_0/β.

Proof:
Setting Ṫ = 0:
-βg + γ(T_0 - T*) = 0
γ(T_0 - T*) = βg
T* = T_0 - (β/γ)g

Uniqueness: Ṫ is linear in T, so exactly one zero.
Stability: ∂Ṫ/∂T = -γ < 0, hence stable.
□
```

Theorem 1 (Inversion Theorem)

Plain Text

```
Statement: Define effective deterrence F(g) = g·[T*(g)]^κ.
There exists a unique critical threshold
g* = T_0γ/[β(κ+1)]
such that:
(i)  F'(g) > 0  for g < g*  (normal regime)
(ii) F'(g) < 0  for g > g*  (inversion regime)

Proof:
F(g) = g(T_0 - βg/γ)^κ

Let c = β/γ. Then F(g) = g(T_0 - cg)^κ

F'(g) = (T_0 - cg)^κ + g·κ(T_0 - cg)^{κ-1}·(-c)
      = (T_0 - cg)^κ - κcg(T_0 - cg)^{κ-1}
      = (T_0 - cg)^{κ-1}[(T_0 - cg) - κcg]
      = (T_0 - cg)^{κ-1}[T_0 - cg(κ+1)]

Setting F'(g) = 0:
T_0 - cg(κ+1) = 0
g* = T_0/[c(κ+1)] = T_0γ/[β(κ+1)]
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
hence F'(g) > 0
hence F'(g) < 0
```

```
Uniqueness: F'(g) is monotonically decreasing (F''(g) < 0).
□
```

Proposition 2 (Behavioral Equilibrium)

```
Plain Text

Statement: At interior equilibrium x* ∈ (0,1):
x*(g) = [F(g) - θ]/α
where F(g) = g[T*(g)]^κ.

This equilibrium is unstable (saddle point).

Proof:
[Complete eigenvalue analysis showing ∂ẋ/∂x|_{x*} > 0]
□
```

Proposition 3 (Comparative Statics)

```
Plain Text

Statement: The critical threshold g* satisfies:
(i)   ∂g*/∂T_0 > 0    (higher baseline trust → higher threshold)
(ii)  ∂g*/∂γ > 0      (faster restoration → higher threshold)
(iii) ∂g*/∂β < 0      (faster erosion → lower threshold)
(iv)  ∂g*/∂κ < 0      (stronger curvature → lower threshold)

Proof: Direct differentiation of g* = T_0γ/[β(κ+1)]
□
```

Action Items:

- ◯ Write all proofs in complete detail
- ◯ Check all algebraic steps
- ◯ Add lemmas where needed (e.g., for stability analysis)
- ◯ Cross-reference with simulation to verify

Day 5-7: Stage 2 Theorems + Bifurcation Analysis

nalysis, prove existence conditions

ce)

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
Plain Text

Statement: When enforcement adapts via Ġ = ψ(x - x_target),
the 3D system exhibits a Hopf bifurcation at critical parameter values.

Specifically, there exists a curve in (ψ, g_target) space where:
(i)   A pair of complex conjugate eigenvalues crosses imaginary axis
(ii)  Transversality condition holds
(iii) Limit cycles emerge for parameters beyond the bifurcation curve

Conditions: [Derive explicit inequalities]
```

Action Items:

- ◯ Compute full Jacobian matrix $J(x^*, T^*, G^*)$ for 3D system
- ◯ Derive characteristic polynomial $\det(J - \lambda I) = 0$
- ◯ Solve for eigenvalues as function of parameters
- ◯ Find curve where $\mathrm{Re}(\lambda_{1,2}) = 0$, $\mathrm{Im}(\lambda_{1,2}) \neq 0$
- ◯ Verify transversality: $d(\mathrm{Re}\,\lambda)/d\psi|_{\psi=\psi_c} \neq 0$
- ◯ Numerical verification via simulation

Tools: Use SymPy for symbolic computation of characteristic polynomial

---

Week 2: Computational Implementation + Visualization

Goal: Create publication-quality figures demonstrating all theoretical results

Day 1-2: Stage 1 Visualizations

Deliverable: Python notebook generating all Stage 1 figures

Figures to Create:

1. **Inversion Curve** (like your HTML version but publication quality)
    - Plot F(g) vs g showing hump shape
    - Mark g* with vertical line
    - Shade normal/inversion regimes
    - Add T*(g) as secondary curve
2. **Phase Portrait** (x, T) space

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

itial conditions

- Fixed points marked

3. **Bifurcation Diagram** (x* vs g)
   - Show stable branches (x=0, x=1)
   - Show unstable interior branch x*(g)
   - Mark g* where maximum occurs
4. **Comparative Statics Panel**
   - 2×2 grid showing effect of varying T_0, β, γ, κ
   - Each subplot shows F(g) curve with different parameters
   - Demonstrate g* shifts

Code Structure:

Python

```python
import numpy as np
import matplotlib.pyplot as plt

class Stage1Model:
    def __init__(self, theta, alpha, sigma, beta, gamma, T0, kappa):
        self.theta = theta
        self.alpha = alpha
        # ... store all parameters

    def T_star(self, g):
        """Equilibrium trust"""
        return self.T0 - (self.beta/self.gamma) * g

    def F(self, g):
        """Effective deterrence"""
        T = self.T_star(g)
        return g * T**self.kappa

    def g_star(self):
        """Critical threshold"""
        return self.T0 * self.gamma / (self.beta * (self.kappa + 1))

    def x_star(self, g):
        """Interior equilibrium"""
        return (self.F(g) - self.theta) / self.alpha

f):

self.g_max, 500)
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
# Usage:
model = Stage1Model(theta=0.02, alpha=0.08, ...)
model.plot_inversion_curve()
```

Action Items:

- ◯ Implement complete model class
- ◯ Generate all 4 figure types
- ◯ Export as PDF and PNG (300 dpi)
- ◯ Create captions explaining each figure

Day 3-4: Stage 2 Visualizations

Deliverable: Figures demonstrating 3D dynamics

Figures to Create:

1. **3D Phase Portrait** (x, T, G) space
    - Trajectories spiraling toward limit cycle
    - Fixed point marked
    - Project onto 2D planes (x-T, x-G, T-G)
2. **Time Series** showing oscillations
    - Three panels: x(t), T(t), G(t)
    - Show approach to limit cycle
    - Mark period and amplitude
3. **Bifurcation Diagram** (parameter sweep)
    - x-axis: bifurcation parameter (e.g., ψ)
    - y-axis: max/min of x in limit cycle
    - Show transition from fixed point to oscillations
4. **Basin of Attraction** (2D slice)
    - Color-code initial conditions by outcome
    - Show boundary between different attractors

Action Items:

- ◯ Implement RK4 integrator for 3D system

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

ameter sweep

y

Day 5-7: Interactive Dashboard (Optional but Impressive)

Deliverable: Streamlit or Dash app for exploring model

Features:

- Sliders for all parameters
- Real-time plot updates
- Phase portrait + time series side-by-side
- Export current configuration as JSON

Why This Helps:

- Professor can explore parameter space interactively
- Demonstrates technical competence
- Useful for future presentations

Action Items:

- ◯ Build Streamlit app
- ◯ Deploy locally (don't need hosting for professor meeting)
- ◯ Create quick-start guide

Week 3: Welfare Analysis + Mechanism Design Extension

Goal: Add normative analysis - what SHOULD platform do?

Day 1-3: Welfare Theorems

Task: Define social welfare, prove equilibrium is inefficient

Deliverable: Formal welfare analysis

Social Welfare Function:

```
Plain Text

W(x, T, g) = ∫[u_i - harm(x) - cost(g)]di + platform_value(T)

= -(c_x + δ)·x - c_g·g + v(T) + consumer_surplus(x)

where δ captures externality (harm to non-users)
```

n)

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

Plain Text

```
Statement: The decentralized equilibrium (x*, T*, g*) is Pareto-
inefficient.
Specifically, there exists an allocation (x̂, T̂, ĝ) with:
W(x̂, T̂, ĝ) > W(x*, T*, g*)

Proof:
[Show platform doesn't internalize externality δ]
[Derive first-order conditions for social optimum]
[Compare to equilibrium FOCs, show wedge]
□
```

Quantification:

- Compute welfare loss: $\Delta W = W(\text{optimal}) - W(\text{equilibrium})$
- Express as % of first-best
- Show how it varies with externality size δ

Action Items:

- ◯ Define welfare function precisely
- ◯ Derive social planner's FOCs
- ◯ Solve for constrained-efficient allocation
- ◯ Compute welfare gap numerically

Day 4-7: Optimal Regulation

Task: Design mechanism that implements efficient allocation

Approaches:

1. **Pigouvian Tax/Subsidy**
   - Tax enforcement at rate τ when $g > \hat{g}$
   - Subsidy when $g < \hat{g}$
   - Derive optimal $\tau(g)$
2. **Quantity Regulation**
   - Mandate $g \leq \hat{g}$
   - Require trust monitoring and reporting

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

, transfer_t)}

u

- Incentive compatibility constraints

Theorem 4 (Implementation)

```
Plain Text

Statement: The efficient allocation can be implemented via a tax
τ(g) = δ·∂x*/∂g
applied to enforcement intensity.

Proof: [Show modified FOCs align with social optimum]
□
```

Action Items:

- ◯ Derive optimal tax formula
- ◯ Simulate outcomes under different regulatory regimes
- ◯ Create comparison table (no reg vs. tax vs. quota)

Week 4: Markov Perfect Equilibrium (MPE) Analysis

Goal: Characterize equilibrium without commitment

Day 1-3: MPE Setup and Solution Method

Task: Solve for platform's policy function g*(x, T)

Approach: Solve Hamilton-Jacobi-Bellman (HJB) equation

HJB Equation:

```
Plain Text

ρV(x,T) = max_g {-c_x·x - c_g·g + v(T)
                 + V_x·ẋ + V_T·Ṫ}
```

where ẋ, Ṫ depend on g and current state.

First-Order Condition:

```
Plain Text

-c_g + V_x·∂ẋ/∂g + V_T·∂Ṫ/∂g = 0
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
g*(x,T) solves above FOC
```

Numerical Solution (Finite Difference Method):

Python

```python
def solve_MPE(params, nx=50, nT=50, tol=1e-6):
    """
    Solve for MPE value function V(x,T) and policy g(x,T)

    Uses value function iteration on discretized state space
    """
    # Grid
    x_grid = np.linspace(0, 1, nx)
    T_grid = np.linspace(0, 1, nT)

    # Initialize value function
    V = np.zeros((nx, nT))
    g_policy = np.zeros((nx, nT))

    # Iterate until convergence
    while not converged:
        V_new = np.zeros_like(V)

        for i, x in enumerate(x_grid):
            for j, T in enumerate(T_grid):
                # Optimize over g
                def objective(g):
                    dx = drift_x(x, T, g, params)
                    dT = drift_T(T, g, params)

                    # Compute V_x, V_T via finite difference
                    V_x = (V[i+1,j] - V[i-1,j])/(2*dx_grid)
                    V_T = (V[i,j+1] - V[i,j-1])/(2*dT_grid)

                    flow_payoff = -c_x*x - c_g*g + v(T)
                    continuation = V_x*dx + V_T*dT

                    return -(flow_payoff + continuation/rho)

                res = minimize(objective, x0=0.5, bounds=[(0, g_max)])
                g_policy[i,j] = res.x
                s.fun

                V)) < tol:
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
            break
        V = V_new

    return V, g_policy
```

Action Items:

- ◯ Implement HJB solver
- ◯ Verify convergence
- ◯ Plot policy function g*(x, T) as heatmap
- ◯ Compare to commitment optimum

Day 4-7: Time-Inconsistency Analysis

Task: Quantify welfare loss from lack of commitment

Deliverables:

1. **Numerical Comparison**
   - Simulate under commitment policy
   - Simulate under MPE policy
   - Compute welfare difference
2. **Proposition 5 (Over-Enforcement)**

   Plain Text

   ```
   Statement: The MPE policy exhibits over-enforcement:
   g^{MPE}(x, T) ≥ g^{commitment}(x, T) for all (x, T)

   Intuition: Without commitment, platform over-exploits trust stock
   because it doesn't fully internalize future erosion costs.
   ```

3. **Quantification Table**

   Plain Text

   ```
   Parameter Set | W^commit | W^MPE | Loss (%)
   -------------+-----------+-------+---------
   Baseline     | 0.85      | 0.72  | 15.3%
   High ß       | 0.78      | 0.58  | 25.6%
                              0.48  | 26.2%
   ```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

- ◯ Solve both problems numerically
- ◯ Create comparison plots
- ◯ Derive bounds on welfare loss analytically (if possible)

Week 5: Agent-Based Model (ABM)

Goal: Test when mean-field approximation (ODE) breaks down

Day 1-3: ABM Implementation

Task: Build working ABM that replicates ODE in baseline case

Specification:

Python

```python
import mesa
import networkx as nx

class User(mesa.Agent):
    def __init__(self, unique_id, model, theta, kappa):
        super().__init__(unique_id, model)
        self.theta = theta      # intrinsic appeal (private type)
        self.kappa = kappa      # trust sensitivity (private type)
        self.action = 0         # 0 = refrain, 1 = share
        self.trust = model.T0   # individual trust level

    def step(self):
        # Observe neighborhood
        neighbors = self.model.grid.get_neighbors(self.unique_id)
        x_local = np.mean([n.action for n in neighbors])

        # Update trust
        saw_enforcement = np.random.rand() < self.model.p_observe
        if saw_enforcement:
            self.trust += self.model.dt * (
                -self.model.beta * self.model.g +
                self.model.gamma * (self.model.T0 - self.trust)
            )

        # Update action (logit best response)
        payoff = self.theta + self.model.alpha * x_local \
                 self.model.[illegible] * self.trust**self.kappa

                 .exp(-payoff / self.model.tau))
                 rand() < prob_share
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
class GovernanceModel(mesa.Model):
    def __init__(self, N, network_type, params):
        super().__init__()
        self.N = N
        self.params = params

        # Create network
        if network_type == 'BA':
            G = nx.barabasi_albert_graph(N, m=3)
        elif network_type == 'WS':
            G = nx.watts_strogatz_graph(N, k=6, p=0.1)

        self.grid = mesa.space.NetworkGrid(G)

        # Create agents
        for i in range(N):
            theta = np.random.beta(2, 5)  # Skewed toward low values
            kappa = np.random.uniform(1.5, 3.0)
            agent = User(i, self, theta, kappa)
            self.grid.place_agent(agent, i)

        # Platform enforcement (adaptive)
        self.g = params['g0']

        # Data collection
        self.datacollector = mesa.DataCollector(
            model_reporters={
                "x": lambda m: np.mean([a.action for a in
m.schedule.agents]),
                "T": lambda m: np.mean([a.trust for a in
m.schedule.agents]),
                "g": "g"
            }
        )

    def step(self):
        # Update enforcement (adaptive)
        x_current = np.mean([a.action for a in self.schedule.agents])
        self.g += self.params['psi'] * (x_current -
self.params['x_target'])
        self.g = np.clip(self.g, 0, 2.0)

        # Agents act

ct(self)
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```python
# Run simulation
model = GovernanceModel(N=5000, network_type='BA', params={...})
for _ in range(1000):
    model.step()

# Extract data
df = model.datacollector.get_model_vars_dataframe()
```

Action Items:

- ◯ Implement User and Model classes
- ◯ Verify baseline: ABM matches ODE when network is complete, types homogeneous
- ◯ Create comparison plot (ABM vs ODE time series)

Day 4-7: Robustness Experiments

Task: Test when mean-field breaks

Experiments:

Experiment	Variable	Hypothesis
Network topology	BA vs WS vs ER	Scale-free networks lower g*
Type heterogeneity	Var(θ), Var(κ)	High variance smooths inversion
Trust diffusion	ζ (social influence)	Strong diffusion creates clusters
Information friction	p_observe	Imperfect visibility reduces β

Deliverable: Table of results

Plain Text

```
Condition            | g*_ODE | g*_ABM | Deviation | Interpretation
---------------------+--------+--------+-----------+---------------
                     |        |  .471  | +0.9%     | Match (good)
                     |        |  .423  | -9.4%     | Hubs amplify
                     |        |  .452  | -3.2%     | Clusters buffer
                     |        |  .488  | +4.5%     | Smoothed hump
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
ζ = 0.3 (strong)    | 0.467  | N/A    | Multi-eq  | Breakdown!
```

Action Items:

- ◯ Run all experiments (20 parameter combinations × 10 replicates)
- ◯ Statistical analysis (t-tests for significant deviations)
- ◯ Create summary table and interpretation

Week 6: Integration + Professor Presentation Prep

Goal: Assemble all pieces, create presentation materials

Day 1-2: Master Document Assembly

Deliverable: Complete PDF containing all results

Structure:

Plain Text

```
1. Game-Theoretic Setup (5 pages)
   - Formal definitions
   - Derivation of dynamics

2. Stage 1: Analytical Results (8 pages)
   - Theorems 1-3 with proofs
   - Figures 1-4

3. Stage 2: Dynamic Equilibrium (6 pages)
   - Hopf bifurcation theorem
   - Limit cycle analysis
   - Figures 5-7

4. Welfare and Policy (5 pages)
   - Inefficiency theorem
   - Optimal regulation
   - MPE vs commitment

5. Agent-Based Validation (4 pages)
   - Model specification
   - Baseline validation
   - Robustness results
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
    - ABM code
```

Action Items:

- ◯ Compile all LaTeX sections
- ◯ Cross-reference all equations and figures
- ◯ Check notation consistency
- ◯ Generate bibliography

Day 3-4: Presentation Slides

Deliverable: 20-minute presentation for professor

Slide Outline (15-20 slides):

1. **Motivation** (2 slides)
    - Real-world puzzle: Twitter moderation escalation
    - Preview of mechanism
2. **Model Setup** (3 slides)
    - Game specification
    - Equilibrium concept
    - Key equations (visual, minimal math)
3. **Main Results** (5 slides)
    - Theorem 1: Inversion exists (show F(g) curve)
    - g* formula and interpretation
    - Comparative statics (show 4-panel figure)
    - Welfare loss quantification
    - MPE time-inconsistency
4. **Dynamics** (3 slides)
    - Stage 2: Hopf bifurcation
    - Limit cycles (show 3D trajectory)
    - Bifurcation diagram
5. **Robustness** (2 slides)
    - ABM validation

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

    - Empirical predictions
7. **Next Steps** (1 slide)
    - Empirical calibration plan
    - Extensions to consider

Design Tips:

- Maximum 1 equation per slide
- Use visual proofs where possible (geometric intuition)
- Highlight key results in color boxes
- Include "backup slides" with full proofs

Action Items:

- ◯ Create Beamer slides (or PowerPoint if preferred)
- ◯ Practice timing (aim for 18 minutes + 2 min buffer)
- ◯ Prepare answers to likely questions

Day 5-7: Code Review and Documentation

Task: Make all code reproducible and documented

Deliverables:

1. **GitHub Repository** (or similar)

Plain Text

```
/governance-inversion/
├── README.md                  # Quick start guide
├── requirements.txt           # Dependencies
├── /stage1/
│   ├── model.py              # Stage 1 class
│   ├── figures.py            # Generate all figures
│   └── theorems.ipynb        # Numerical verification
├── /stage2/
│   ├── model.py              # 3D system
│   ├── bifurcation.py        # Hopf analysis
│   └── visualize.py          # 3D plots
├── /mpe/
│   ├── hjb_solver.py         # PDE solver
│   └── policy_comparison.py  # MPE vs commitment
├── /abm/
                              # Mesa model
                              # Run all robustness tests
                              # Generate results table

                             # Master document
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

```
└── figures/              # All generated figures
```

2. **README with Instructions**

Markdown

Trust-Mediated Governance Inversion

Reproducing Results

Stage 1 (Analytical)

python stage1/figures.py —output paper/figures/

Plain Text

```
### Stage 2 (Bifurcation)
```

python stage2/bifurcation.py —params baseline

Plain Text

```
### MPE Solution
```

python mpe/hjb_solver.py —nx 100 —nT 100

Plain Text

```
### ABM Experiments
```

N 5000 —reps 10

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

3. **Jupyter Notebook Demo**
    - Interactive walkthrough of main results
    - Professor can run cells and see outputs
    - Useful for exploring parameter sensitivity

Action Items:

- ◯ Organize all code into clean structure
- ◯ Add docstrings to all functions
- ◯ Create requirements.txt
- ◯ Write comprehensive README
- ◯ Test that everything runs from scratch

Deliverables Summary (What to Show Professor)

Document 1: Formal Theory (PDF, ~30 pages)

- ✅ Game-theoretic setup
- ✅ All theorems with complete proofs
- ✅ Bifurcation analysis
- ✅ Welfare results

Document 2: Computational Results (PDF, ~15 pages)

- ✅ All figures (publication quality)
- ✅ Phase portraits
- ✅ Bifurcation diagrams
- ✅ MPE policy functions
- ✅ ABM robustness table

Document 3: Code Repository

- ✅ Organized, documented, reproducible
- ✅ README with clear instructions
- ✅ All scripts run without errors

Presentation: 20-minute slides

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

Critical Path and Priorities

Must Have (Non-Negotiable):

1. ✅ Complete proofs for Theorems 1-3 (Stage 1)
2. ✅ Inversion curve figure (F(g) showing hump)
3. ✅ Hopf bifurcation analysis (even if not fully rigorous)
4. ✅ Basic ABM showing validation

Should Have (Very Important):

1. ✅ MPE solution (at least numerically)
2. ✅ Welfare analysis
3. ✅ All Stage 1 figures (phase portrait, bifurcation, comp statics)
4. ✅ Clean code repository

Nice to Have (If Time):

1. ⚪ Interactive dashboard
2. ⚪ Full analytical characterization of Hopf conditions
3. ⚪ Extensive ABM experiments (can show 2-3 instead of 10)

Daily Checklist Template

Use this to track progress:

Plain Text

```
Week 1, Day 1: Game Setup
[ ] Write user payoff function formally
[ ] Derive replicator dynamics
[ ] Write platform objective
[ ] Draft 3-page setup document

Week 1, Day 2: Complete setup
[ ] Add trust microfoundation
[ ] Verify all notation consistent
[ ] Create summary table of variables
```

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

Risk Mitigation

Potential Bottlenecks:

Risk 1: MPE solver doesn't converge

- **Mitigation**: Start with coarse grid (20×20), increase gradually
- **Backup**: Show commitment solution only, note MPE is work in progress

Risk 2: Hopf conditions too complex to derive analytically

- **Mitigation**: Numerical characterization is acceptable
- **Backup**: State theorem numerically ("we find bifurcation at $\psi = 0.15$")

Risk 3: ABM takes too long to run

- **Mitigation**: Reduce N from 10,000 to 1,000 for experiments
- **Backup**: Show 3 key experiments instead of 10

Risk 4: Proofs have gaps

- **Mitigation**: Start with proof sketches, fill in details iteratively
- **Backup**: Label as "Proof Sketch" and note full proof in appendix

---

Questions to Prepare for Professor

Likely questions and how to answer:

Q1: "Why is this game-theoretic and not just a dynamical system?"
A: "Users are strategic agents with payoffs who best-respond. Platform optimizes intertemporally. Trust evolves endogenously from strategic interactions. These are equilibrium dynamics, not arbitrary ODEs."

Q2: "How do you know MPE exists and is unique?"
A: "We verify numerically that value function iteration converges. For existence proof, we'd invoke standard results from Ljungqvist-Sargent. Uniqueness we check via sensitivity analysis—small perturbations in initial guess lead to same solution."

Q3: "What's the empirical content?"
A: "Model predicts: (1) non-monotonic enforcement effects, (2) cross-sectional variation in g* correlates with trust proxies, (3) time-series oscillations under adaptive governance. We plan to test against Twitter 2020-21 data."

Q4: "Isn't the ABM just confirming your ODE?"
A: "No—it reveals boundary conditions. When trust diffusion is strong or networks
own and we see emergent patterns not in the
odel is valid."

g work?"
rust as institutional capital, (2) multiplicative

Update Available
Version 0.54.2 is ready to be installed
Later
Update App

trust-enforcement coupling, (3) analytical derivation of structural instability threshold, (4) game-theoretic microfoundations for enforcement-trust feedback. See literature review document for detailed comparison."

Week-by-Week Milestones

End of Week 1: Have all theorems stated and proven (at least sketch-level)

End of Week 2: Have all figures generated, code working

End of Week 3: Have welfare analysis complete

End of Week 4: Have MPE solved numerically

End of Week 5: Have ABM validated and robustness tested

End of Week 6: Have complete presentation ready

After Professor Meeting: Next Steps

Depending on feedback, you'll likely proceed to:

1. **Empirical Calibration** (Weeks 7-8)
    - Fit model to Twitter transparency reports
    - Estimate parameters via SMM or MLE
    - Test predictions
2. **Paper Writing** (Weeks 9-12)
    - Draft all sections
    - Incorporate feedback
    - Polish figures
3. **Submission Prep** (Weeks 13-14)
    - Final revisions
    - Appendices
    - Cover letter

But that comes after you've shown solid theoretical and computational results.

Update Available
Version 0.54.2 is ready to be installed
Later
Update App